PYTHON PROGRAMMING
CASE STUDY

ATM Transaction Sequence Simulation Using Python

Problem No. 13 | Medium Difficulty

Name: _____

Roll Number: _____

Branch: CSE (AI/ML)

College: _____

Subject: Python Programming

Python Concepts: Loops, Lists, Conditions and Arithmetic

1. Title

ATM Transaction Sequence Simulation Using Python

2. Problem Statement

An Automated Teller Machine (ATM) allows users to perform banking transactions such as depositing money and withdrawing money. The aim of this program is to simulate a sequence of ATM transactions using Python.

The program should accept an initial account balance and process multiple transactions entered by the user. It should allow valid deposits and withdrawals, reject invalid transactions, and maintain the correct account balance throughout the process.

At the end, the program should display the final account balance and a summary of all transactions performed. The program uses Python concepts such as loops, lists, conditional statements, and arithmetic operations.

3. Objectives

1. To understand how ATM transactions work using Python.
2. To accept deposits and withdrawals from the user.
3. To maintain and update the account balance correctly.
4. To reject invalid transactions, such as negative amounts or withdrawals exceeding the available balance.
5. To store transaction details using Python lists.
6. To use loops and conditional statements to process multiple transactions.
7. To display the final balance and a transaction summary.

4. Assumptions

8. The initial account balance must be zero or a positive amount.
9. The user enters the number of transactions to be processed.
10. Each transaction is either a deposit or a withdrawal.
11. Transaction amounts must be greater than zero.
12. A withdrawal is accepted only when sufficient balance is available.
13. Invalid transactions are rejected without changing the account balance.
14. All transaction details are stored in a list.
15. This is a basic simulation for educational purposes, not a real banking application.

5. Algorithm

16. Start the program.
17. Read the initial account balance from the user.
18. Read the number of transactions to be processed.
19. Create an empty list to store transaction details.
20. Repeat the following steps for each transaction:
21. Ask the user to enter the transaction type: deposit or withdrawal.
22. Ask the user to enter the transaction amount.

23. If the amount is zero or negative, reject the transaction.
24. If the transaction is a deposit, add the amount to the balance.
25. If the transaction is a withdrawal, check whether sufficient balance is available.
26. If sufficient balance is available, subtract the amount; otherwise, reject the withdrawal.
27. Store the transaction details and its status in the list.
28. Display all transaction details.
29. Display the total number of successful and rejected transactions.
30. Display the final account balance.
31. Stop the program.

6. Flowchart

Use the flowchart below to understand the logical flow. The diagram is represented as a sequence of steps; when drawing it by hand, use an oval for Start/Stop, a parallelogram for input/output, a rectangle for processing, and a diamond for decisions.

START
Enter initial balance and number of transactions
Initialize transaction list and counters
Are transactions remaining?
YES: Enter transaction type and amount
Is amount greater than zero?
NO: Reject transaction
Deposit? Add amount to balance
Withdrawal? Check available balance
If enough balance, subtract amount; otherwise reject
Store transaction details and update counters; repeat loop
When no transactions remain: display summary and final balance
STOP

Flow: START ↓ Input details ↓ Initialize list/counters ↓ Repeat each transaction ↓ Validate amount/type/balance ↓ Store result ↓ Display summary ↓ STOP

7. Pseudocode

```
BEGIN
    INPUT initial_balance
    INPUT number_of_transactions

    SET balance = initial_balance
    CREATE empty transaction_list
    SET successful = 0
    SET rejected = 0

    FOR each transaction from 1 to number_of_transactions
        INPUT transaction_type
        INPUT amount

        IF amount <= 0 THEN
            SET status = "Rejected"
        ELSE IF transaction_type is "deposit" THEN
            ADD amount to balance
            SET status = "Successful"
        ELSE IF transaction_type is "withdrawal" THEN
            IF amount <= balance THEN
                SUBTRACT amount from balance
```

```
                SET status = "Successful"
            ELSE
                SET status = "Rejected"
            END IF
        ELSE
            SET status = "Rejected"
        END IF

        STORE transaction details in transaction_list

        IF status is "Successful" THEN
            INCREASE successful by 1
        ELSE
            INCREASE rejected by 1
        END IF
    END FOR

    DISPLAY transaction_list
    DISPLAY successful transactions
    DISPLAY rejected transactions
    DISPLAY final balance
END
```

8. Python Actual Code

Copy and run the following code in Python IDLE, VS Code, or another Python environment.

```
transactions = []

balance = float(input("Enter initial balance: ₹"))

while balance < 0:
    print("Balance cannot be negative.")
    balance = float(input("Enter initial balance: ₹"))

n = int(input("Enter number of transactions: "))

successful = 0
rejected = 0

for i in range(n):
    print("\nTransaction", i + 1)

    t_type = input("Enter type (deposit/withdrawal): ").lower()
    amount = float(input("Enter amount: ₹"))

    if amount <= 0:
        status = "Rejected"
        print("Invalid amount!")

    elif t_type == "deposit":
        balance += amount
        status = "Successful"
        print("Deposit successful.")

    elif t_type == "withdrawal":
        if amount <= balance:
            balance -= amount
            status = "Successful"
            print("Withdrawal successful.")
        else:
            status = "Rejected"
            print("Insufficient balance!")
```

```
        else:
            status = "Rejected"
            print("Invalid transaction type!")

        if status == "Successful":
            successful += 1
        else:
            rejected += 1

        transactions.append(
            [t_type, amount, status, balance]
        )

print("\n----- TRANSACTION SUMMARY -----")

for i, transaction in enumerate(transactions, start=1):
    print(
        f"{i}. Type: {transaction[0]}, "
        f"Amount: ₹{transaction[1]:.2f}, "
        f"Status: {transaction[2]}, "
        f"Balance: ₹{transaction[3]:.2f}"
    )

print("\nSuccessful transactions:", successful)
print("Rejected transactions:", rejected)
print(f"Final balance: ₹{balance:.2f}")
```

9. Explanation of the Code

9.1 Initializing the transaction list

transactions = [] creates an empty list to store the type, amount, status, and balance after each transaction.

9.2 Reading and validating the initial balance

input() accepts data from the user, and float() converts it to a decimal number. The while loop repeatedly asks for the balance if it is negative.

9.3 Processing multiple transactions

The for loop runs the required number of times and processes each transaction one at a time.

9.4 Checking the transaction amount

The condition if amount <= 0 rejects zero or negative transaction amounts.

9.5 Processing deposits

When the type is deposit, balance += amount adds the deposit to the existing balance.

9.6 Processing withdrawals

The program checks whether the withdrawal amount is less than or equal to the available balance. If enough money is available, it deducts the amount; otherwise, it rejects the withdrawal.

9.7 Storing transaction details

The append() method adds each transaction record to the list, including its type, amount, status, and resulting balance.

9.8 Counting successful and rejected transactions

Conditional statements increase the successful counter for completed transactions and the rejected counter for invalid transactions.

9.9 Displaying the transaction summary

The enumerate() function numbers each record while the program displays transaction details. The final statements show the number of successful and rejected transactions and the final balance.

Python Concepts Used

Python concept	Purpose
Variables	Store the balance, amount, and counters.
for loop	Process multiple transactions.
while loop	Validate the initial balance.
if-elif-else	Check transaction conditions.
Lists	Store transaction records.
append()	Add transaction details to the list.
Arithmetic operators	Add deposits and subtract withdrawals.
enumerate()	Display numbered transaction records.
input() and print()	Accept and display information.

10. Output Screenshot

Paste your actual output screenshot in the space below after running the Python program in IDLE, VS Code, or another Python environment.

[PASTE OUTPUT SCREENSHOT HERE]

Figure 1: Output of ATM Transaction Sequence Simulation

11. Conclusion

The ATM Transaction Sequence Simulation demonstrates how Python can be used to manage a sequence of banking transactions. The program accepts deposits and withdrawals, updates the account balance, rejects invalid transactions, and displays a complete transaction summary.

Through this case study, we understand the practical use of loops, lists, conditional statements, and arithmetic operations in Python. This project also improves problem-solving and logical-thinking skills by applying programming concepts to a real-world banking scenario.

The program is a simple educational model of ATM transaction processing and can be extended in the future by adding features such as PIN verification, daily withdrawal limits, and transaction history stored in a file.

Submission Checklist

- Add your name, roll number, branch, and college details on the first page.
- Check that the algorithm, flowchart, pseudocode, and Python code are included.
- Run the program and paste your actual output screenshot in Section 10.
- Check page numbers, headings, and formatting.
- Save or export the completed document as a PDF if required.